

Astro Website QA in CI/CD

A reusable, automated pre-live QA gate for Astro websites. QA runs on every push, reports PASS / WARNING / FAIL in plain language, and blocks production deployment when critical checks fail.

ENGINE

Node 20 · Playwright · axe-core · Lighthouse CI

DEPLOYMENT

Cloudflare Pages via Wrangler

FIRST REPO

60–90 minutes

CI

GitLab CI and GitHub Actions

SCOPE

Astro sites of roughly 5–100 pages

EACH REPO AFTER

About 10 minutes

Contents

1 What this system does

The problem it solves
What it checks — and what it does not
The pipeline
The severity model

2 What is in the kit

3 Phase 1 — Local setup

Create the branch
Install the kit
Install the dependencies
Run it against the site

4 Configuration reference

Identity — the highest-value checks
Contact details
Pages
Selectors and landmarks
Viewports and layout rules
Interactions
Typography and brand colours
Images
Accessibility
Crawl, runtime noise and timeouts
Lighthouse
Severity overrides
Environment variables

5 Getting to a clean first run

6 Phase 2A — GitLab implementation

7 Phase 2B — GitHub implementation

8 Phase 3 — Cloudflare

9 Phase 4 — Proving the gate works

10 Day-to-day workflow

11 Rolling it out across every site

12 Troubleshooting

A Appendix A — every check, with default severity

B Appendix B — the complete qa-config.json

1 What this system does

1.1 The problem it solves

Most things that embarrass a web team on launch day are not subtle. A link to a page that was renamed. A hero image that 404s only on the live host. The previous client's name still in the footer because the site was cloned from theirs. A contact form that silently disappeared in a refactor. A layout that scrolls sideways on a phone.

All of those are mechanically detectable, and all of them are routinely missed because checking them by hand across five viewports and twenty pages is tedious work that gets skipped under deadline. This system does that work on every push, in about eight minutes, and refuses to let a broken build reach production.

1.2 What it checks — and what it does not

It does *not* read your `.astro` source files. It builds the site, serves the production output, and drives a real browser over the rendered pages across Chromium, Firefox and WebKit.

That distinction matters. A linter reads your components; this reads the website your client will see. It will not tell you a component is badly structured. It will tell you that the structure produced a page with no H1, a stretched logo and 1,441 pixels of horizontal overflow at mobile width.

Thirty-four checks run in seven modules. Appendix A lists all of them. In summary:

MODULE	COVERS
Pages & SEO	Required pages load, the custom 404 page is intact, title / meta description / H1 / viewport / lang, accidental <code>noindex</code> , canonical and Open Graph, heading hierarchy, <code>robots.txt</code> , sitemap, browser runtime errors
Identity	Forbidden business names anywhere on the page, expected business name present, phone and email in both visible text and <code>tel:</code> / <code>mailto:</code> links, privacy policy exists and is linked, logo links to the homepage
Forms	A form exists on every page that needs one (native, GoHighLevel/LeadConnector, HubSpot or other iframe embeds), required fields present, accessible labels, a submit button, and no wrong company name inside the form region
Links	Internal links resolve including soft-404 detection; external links reachable, with rate-limited hosts skipped rather than failed
Images	Failed loads, zero natural size, broken background images, stretched aspect ratios, missing <code>alt</code> , file-size budget, lazy loading below the fold
Layout	Horizontal overflow, disappearing elements, clipped text and element overlap at five viewports, container width and page gutters, optional screenshot regression
Interaction & a11y	Menus, modals, dropdowns and drawers open and close and stay keyboard reachable; hover and focus states produce a visible change; axe-core WCAG 2.1 A/AA scan
Performance	Lighthouse CI on key pages with configurable thresholds

DESIGN PRINCIPLE

Checks that cannot be inferred from the site — brand colour rules, element-overlap pairs, visual-regression baselines — ship **disabled**. Turning them on is a deliberate decision you make once a design is signed off. A QA gate that cries wolf in week one gets switched off in week two.

1.3 The pipeline

- Developer pushes to any branch
- CI installs dependencies and Playwright browsers
- Astro production build
- Local production preview server starts
- Site crawled **once**; all modules share that crawl
- Pages & SEO → links → images → responsive → interactions → accessibility → cross-browser
- Lighthouse CI
- Reporter aggregates everything and decides PASS or FAIL
 - **FAIL** → job fails → deployment never starts
 - **PASS** + push to the default branch → Cloudflare Pages deployment

Two details are worth understanding, because they explain most of the system's behaviour.

Every QA module exits 0. Modules record findings; they never fail the job themselves. A module that crashes records that crash as an error and the rest still run. This means one broken check cannot hide the other thirty-three, and a CI run always produces a complete report.

The reporter is the only gate. `qa-reporter.mjs` reads every module's output, applies the configured severities, writes the reports and exits 1 if any error exists. There is exactly one place where PASS becomes FAIL.

1.4 The severity model

LEVEL	EFFECT	TYPICAL MEMBERS
ERROR	Fails the job, blocks deployment	Broken required page, broken internal link, missing form, wrong company name, horizontal overflow, missing H1, accidental <code>noindex</code> , critical accessibility violation, Lighthouse below threshold
WARNING	Visible in the report, does not block	Missing <code>alt</code> , multiple H1, external link failure, missing canonical, typography or colour drift, oversized image, minor accessibility violation
PASS	Requirement satisfied	—

Every one of the fifty-plus rules can be moved between those levels, or set to `"off"`, from a single `severity` block in the config. Section 4.12 lists every key. Setting `"failOnWarnings": true` makes warnings block as well.

2 What is in the kit

FILE	PURPOSE
<code>scripts/qa-config.json</code>	The only file you edit per site. Business identity, pages, selectors, thresholds, severities.
<code>scripts/qa-lib.mjs</code>	Shared helpers: config loading, severity resolution, the result model, URL normalisation, browser launch, runtime collectors.
<code>scripts/qa-crawl.mjs</code>	One shared crawl, cached to <code>qa-results/crawl.json</code> . Every module reuses it, so the site is crawled once per run, not once per module.
<code>scripts/qa-layout.mjs</code>	Shared layout audit used by the responsive and cross-browser modules so both report identically.
<code>scripts/qa-check.mjs</code>	Core module: required pages, 404 page, SEO, identity, contact details, privacy policy, logo, forms, typography, colours, <code>robots.txt</code> , sitemap, runtime errors.
<code>scripts/check-links.mjs</code>	Internal and external link validation.
<code>scripts/check-images.mjs</code>	Broken, distorted, oversized and un-lazy-loaded images; missing <code>alt</code> .
<code>scripts/responsive-check.mjs</code>	Five viewports, layout audit, screenshots of every failure.
<code>scripts/interaction-check.mjs</code>	Configured interactions plus hover and focus state checks.
<code>scripts/accessibility-check.mjs</code>	axe-core WCAG scan.
<code>scripts/cross-browser-check.mjs</code>	Firefox and WebKit smoke suite.
<code>scripts/qa-reporter.mjs</code>	The gate. Writes <code>qa-report.md</code> , <code>qa-report.json</code> and <code>qa-report.junit.xml</code> ; exits 0 or 1.
<code>scripts/qa-all.mjs</code>	Runs the whole suite locally in CI order. Cross-platform.
<code>scripts/record-result.mjs</code>	Folds an external CI step's outcome into the report.
<code>tests/cross-browser.spec.mjs</code>	Optional Playwright-runner smoke suite, for local debugging with traces.
<code>tests/visual/visual.spec.mjs</code>	Optional visual regression against approved baselines.
<code>lighthouserc.cjs</code>	Lighthouse CI config; reads pages and thresholds from <code>qa-config.json</code> .
<code>playwright.config.mjs</code>	Config for the optional test-runner suites only.
<code>.gitlab-ci.yml</code>	GitLab pipeline.
<code>.github/workflows/qa-deploy.yml</code>	GitHub Actions workflow.
<code>install-qa.mjs</code>	Installs the kit into an Astro project and patches <code>package.json</code> and <code>.gitignore</code> .
<code>README.md</code>	Reference manual.
<code>SETUP-GITHUB.md</code> / <code>SETUP-GITLAB.md</code>	Platform runbooks.

Output produced by a run

ARTIFACT	AUDIENCE
<code>qa-report.md</code>	Project managers. Rendered directly into the GitHub job summary. Tables of errors, warnings, Lighthouse scores and an accessibility summary.
<code>qa-report.json</code>	Tooling. Every finding with module, rule key, page, viewport and severity.
<code>qa-report.junit.xml</code>	CI test widgets. GitLab renders it in the merge request: errors as failed tests, warnings as skipped.
<code>qa-results/screenshots/</code>	Developers. A full-page screenshot of every failed viewport.
<code>.lighthouseci/</code>	Full Lighthouse HTML reports.

3 Phase 1 — Local setup

Identical on GitLab and GitHub. Get a clean run on your machine before you push anything — debugging forty findings through a CI log is slow and demoralising, and the local loop takes seconds.

1 Create the branch

```
git checkout main && git pull
git checkout -b setup/qa-cicd
```

2 Install the kit

Run from the root of the Astro project. The `.` is the target directory.

```
node /path/to/astro-qa-kit/install-qa.mjs .          # GitHub Actions
node /path/to/astro-qa-kit/install-qa.mjs . --gitlab # GitLab CI
node /path/to/astro-qa-kit/install-qa.mjs . --both  # both CI files
```

The installer copies the engine, the CI file, `lighthouse.cjs`, `playwright.config.mjs` and the test suites; adds the `qa:*` npm scripts; and appends the QA output paths to `.gitignore`. It never overwrites an existing file — pass `--force` only when deliberately upgrading a site that already has the kit.

```
git status --short
```

Expect `scripts/`, `tests/`, `lighthouse.cjs`, `playwright.config.mjs`, the CI file, and a modified `package.json` and `.gitignore`.

3 Install the dependencies

```
npm install -D playwright @playwright/test @lhci/cli wait-on wrangler @axe-core/playwright
npx playwright install --with-deps chromium firefox webkit
```

The browser download is roughly 400 MB and happens once per machine. On Windows and macOS, drop `--with-deps`; it installs Linux system packages and does nothing elsewhere.

LOCAL SHORTCUT

To skip the browser download while experimenting, run against an installed Chrome or Edge:

```
QA_CHROMIUM_CHANNEL=chrome npm run qa:all .
```

Leave this unset in CI so every run uses the pinned Playwright build.

4 Run it against the site

Terminal 1 — build and serve the production output:

```
npm run build && npm run preview -- --host 127.0.0.1 --port 4321
```

Terminal 2 — run the suite:

```
npm run qa:all
```

While iterating on the config, skip the two slow modules:

```
npm run qa:all -- --skip lighthouse,crossbrowser
```

Individual modules are available as `npm run qa:core`, `qa:links`, `qa:images`, `qa:responsive`, `qa:interactions`, `qa:accessibility` and `qa:cross-browser`. Each writes to `qa-results/` and exits 0; `npm run qa:report` aggregates them and is the only command that can exit non-zero.

SSR SITES

`astro preview` does not work with an SSR adapter. Serve the built output with the adapter's own command instead and point the QA engine at it with `QA_BASE_URL`.

4 Configuration reference

Everything site-specific lives in `scripts/qa-config.json`. The engine contains no client names and no production URLs, which is what makes the same code reusable across every site you build. Appendix B is the complete default file.

4.1 Identity — the highest-value checks

```
"businessName": "Acme Plumbing",
"businessNameAliases": ["Acme Plumbing Ltd", "Acme"],
"forbiddenBusinessNames": ["Nextkin", "Previous Client Name", "Lorem Ipsum Co"]
```

`forbiddenBusinessNames` is the single most valuable field in the file. Fill it with the name of whatever site this one was cloned from, plus any placeholder names your templates carry. The engine scans visible text, `<title>`, meta description, Open Graph tags, the footer and form regions. A hit is an error, which is exactly right: a leaked client name must never reach production.

`businessName` missing from a required page is a **WARNING** by default, since a legitimate page may not mention it.

4.2 Contact details

```
"phone": "+1 604 555 1234",
"requiredPhone": true,
"email": "hello@acmeplumbing.com",
"emailDomain": "acmeplumbing.com",
"requiredEmail": true
```

Phone numbers are compared on their last ten digits, so `(604) 555-1234`, `604.555.1234` and `+16045551234` all match. The engine checks both the visible text and the `tel:` links, and reports three distinct situations: missing entirely (**ERROR** when required), present as text but with no matching `tel:` link or vice versa (**WARNING**), and a `tel:` link pointing at a completely different number (**WARNING**). The last one catches a very common template-leftover bug.

`emailDomain` flags `mailto:` links on an unexpected domain.

4.3 Pages

```
"requiredPages": ["/", "/about/", "/services/", "/contact/", "/privacy-policy/"],
"formPages": ["/contact/", "/request-a-quote/"],
"responsivePages": ["/", "/services/", "/contact/"],
"lighthousePages": ["/", "/about/", "/services/", "/contact/"],
"privacyPolicyUrl": "/privacy-policy/",
"notFoundPath": "/qa-404-probe-do-not-create/"
```

KEY	MEANING
<code>requiredPages</code>	Must return 2xx (redirects allowed). Also seeds the crawl.
<code>formPages</code>	Must contain a valid form.
<code>responsivePages</code>	Tested at every viewport. Keep short — this is pages × viewports.
<code>lighthousePages</code>	Lighthouse runs here, twice each by default.
<code>notFoundPath</code>	A URL guaranteed not to exist. The engine requests it to confirm a real 404 status and an intact 404 page: heading, body content, header, navigation links, footer, no broken images, no overflow.

4.4 Selectors and landmarks

```
"logoSelector": "header a.logo",
"headerSelector": "header",
"footerSelector": "footer",
"navSelector": "header nav",
"formSelectors": ["form", "iframe[src*='leadconnectorhq']", "iframe[src*='hubspot']"],
"formRequiredFields": ["name", "email"]
```

Open the site and confirm each selector matches before the first run. A wrong `logoSelector` produces a baffling "logo not found" error on every single run and is the most common setup mistake.

`formRequiredFields` entries are matched loosely against each field's name, accessible label, placeholder and type, so `"email"` matches a field named `contact_email` or labelled "Email address".

EMBEDDED FORMS

Cross-origin iframes cannot be read into. For GoHighLevel, HubSpot and similar embeds the engine verifies the iframe is present and visible, and scans its `title` and `src` for forbidden business names. Field-level checks apply to native `<form>` elements only.

4.5 Viewports and layout rules

```
"viewports": {
  "mobile": { "width": 375, "height": 812 },
  "tablet": { "width": 768, "height": 1024 },
  "laptop": { "width": 1366, "height": 768 },
  "desktop": { "width": 1920, "height": 1080 },
  "wide": { "width": 2560, "height": 1440 }
},
"design": {
  "containerSelector": ".container",
  "containerMaxWidth": 1280,
  "pageGutterMin": 16,
  "overflowTolerancePx": 2,
  "requiredVisibleSelectors": ["header", "main", "footer", "header nav"],
  "noOverlapSelectors": []
}
```

At each viewport the engine scrolls the full page to trigger lazy content, then checks:

- **Horizontal overflow** — `scrollWidth` beyond `clientWidth` past the tolerance, naming the outermost offending elements and their right edge in pixels. **ERROR**.

- **Missing elements** — anything in `requiredVisibleSelectors` absent or invisible. **ERROR**.
- **Clipped text** — text cut off by `overflow: hidden` without an intentional ellipsis. **WARNING**.
- **Overlap** — pairs from `noOverlapSelectors` covering more than 25% of the smaller element. **WARNING**.
- **Container and gutters** — only when `containerSelector` is set. **WARNING**.

`noOverlapSelectors` ships empty on purpose: generic overlap detection produces constant false positives. List only elements that must never collide, for example `["header .logo", "header nav"]`.

Every failed viewport produces a full-page screenshot in `qa-results/screenshots/`, uploaded as a CI artifact.

4.6 Interactions

```
"interactions": [
  {
    "name": "Mobile navigation",
    "page": "/",
    "viewport": "mobile",
    "trigger": "[data-qa='menu-toggle']",
    "target": "[data-qa='mobile-menu']",
    "expect": "visible",
    "closeAfter": true,
    "optional": false,
    "critical": true
  }
]
```

Each entry drives one open-and-close test. The engine clicks the trigger, waits for the target to reach the expected state, then presses Escape and, failing that, activates the trigger again to confirm it closes. It also verifies the trigger is keyboard focusable and that no console errors fire during the interaction.

KEY	EFFECT
<code>optional</code>	<code>true</code> passes quietly when the trigger does not exist. Use while a component is still being built.
<code>critical</code>	Defaults to <code>true</code> ; also runs in the Firefox and WebKit smoke suite.
<code>closeAfter</code>	Set <code>false</code> for things that are not meant to close.
<code>expect</code>	<code>"visible"</code> or <code>"hidden"</code> .

Add one entry per menu, modal, dropdown, accordion, tab set and drawer. If you control the templates, add `data-qa` attributes and target those — they survive CSS refactors in a way that `.nav__toggle--active` does not.

Hover and focus states are configured separately:

```
"states": {
  "enabled": true,
  "pages": ["/"],
  "selectors": ["header nav a", "a.btn", "button[type='submit']"],
  "maxElementsPerSelector": 5,
  "requireFocusIndicator": true
}
```

For each element the engine snapshots computed styles, hovers, compares, moves the pointer away, focuses, and compares again. No change on hover is a warning; no visible focus indicator is a warning and a genuine

keyboard-accessibility problem.

4.7 Typography and brand colours

```
"design": {
  "typography": {
    "body": { "selector": "body", "fontFamily": "Inter", "fontSize": "16px", "lineHeight": "24px" },
    "h1": { "selector": "h1", "fontFamily": "Inter", "fontSize": "48px", "fontWeight": "700" },
    "nav": { "selector": "header nav a", "fontSize": "16px" }
  },
  "brandColors": ["#111827", "#2563EB"],
  "colorRules": [
    { "name": "Primary button", "selector": ".btn-primary", "property": "backgroundColor", "expected": "#2563EB" },
    { "name": "Body text", "selector": "body", "property": "color", "expected": "#111827" }
  ]
}
```

Font families are compared on the first family in the computed stack, so a fallback chain does not cause a false failure. Colours are normalised, so `#2563EB`, `rgb(37, 99, 235)` and `rgba(37,99,235,1)` all compare equal. A rule with no `expected` falls back to the `brandColors` palette.

With `colorRules` empty, colour checking is skipped entirely. Both default to **WARNING**; raise them to **ERROR** once a design is signed off and you want drift to block.

4.8 Images

```
"imageRules": {
  "maxFileSizeKB": 500,
  "requireLazyLoadBelowFold": true,
  "preferModernFormats": false,
  "checkBackgroundImages": true,
  "aspectRatioTolerance": 0.1,
  "maxPages": 50,
  "criticalSelectors": ["header img", "img[alt*='logo' i]", "[class*='hero'] img"]
}
```

Anything matching `criticalSelectors` that fails to render is an **ERROR**; other broken images are a **WARNING** by default. Distortion is only reported when `object-fit` actually allows stretching, which keeps the false-positive rate near zero. Lazy-loading warnings apply only below the fold, so critical LCP images are never flagged. `alt=""` is accepted as a deliberate marker for a decorative image; a missing `alt` attribute is not.

4.9 Accessibility

```
"accessibility": {
  "enabled": true,
  "pages": ["/", "/contact/"],
  "tags": ["wcag2a", "wcag2aa", "wcag21a", "wcag21aa"],
  "disableRules": [],
  "errorImpacts": ["critical", "serious"]
}
```

Violations at an impact listed in `errorImpacts` become errors; everything else is a warning. The report includes a dedicated accessibility summary with a link to axe's fix guidance for each rule. Lighthouse contributes a

second, independent accessibility score on top.

EXPECTATION SETTING

Automated tooling catches roughly a third of real accessibility problems. Keyboard-only navigation and a screen-reader pass are still worth doing before a launch that matters.

4.10 Crawl, runtime noise and timeouts

```
"crawl": {
  "enabled": true,
  "maxPages": 100,
  "concurrency": 5,
  "checkExternal": true,
  "requireExternalBlank": false,
  "excludePatterns": ["^/admin", "^/api/"],
  "softNotFoundText": []
},
"runtime": {
  "ignorePatterns": ["favicon.ico", "google-analytics", "googletagmanager",
    "leadconnectorhq", "hubspot", "ResizeObserver loop"]
},
"timeouts": { "navigationMs": 30000, "requestMs": 15000 }
```

The crawl starts from `requiredPages`, stays on the current origin, normalises URLs so `/about` and `/about/` are one page, skips `mailto:`, `tel:`, `javascript:` and anchors, and stops at `maxPages`.

`softNotFoundText` catches soft 404s: if your 404 page returns HTTP 200 (some hosts do), list a phrase unique to it, such as `"Page not found"`, and the engine will treat pages rendering it as broken.

`runtime.ignorePatterns` suppresses third-party console noise. These are matched against both the message text and the resource URL, so an analytics script failing to load does not fail your build.

4.11 Lighthouse

```
"lighthousePages": ["/", "/about/", "/services/", "/contact/"],
"lighthouse": {
  "enabled": true,
  "numberOfRuns": 2,
  "performance": 0.75,
  "accessibility": 0.90,
  "bestPractices": 0.90,
  "seo": 0.90
}
```

Lighthouse CI runs each page `numberOfRuns` times and the median is used, which absorbs most of the variance from shared CI runners. The reporter reads `.lighthouseci/manifest.json` and applies these thresholds itself, so Lighthouse failures appear in the same report as everything else rather than failing the job separately.

Add `"preset": "desktop"` to score against the desktop profile instead of the default mobile one.

4.12 Severity overrides

```
"severity": {
  "missingAlt": "error",
  "externalLink": "off",
  "typography": "error",
  "accessibilityMinor": "off"
}
```

Values are "error", "warning" or "off". Every rule key:

GROUP	KEYS
Pages	<code>brokenRequiredPage</code> <code>notFoundPage</code> <code>softNotFound</code>
Links	<code>internalLink</code> <code>externalLink</code> <code>externalTargetBlank</code>
Images	<code>criticalImage</code> <code>nonCriticalImage</code> <code>distortedImage</code> <code>backgroundImage</code> <code>imageSize</code> <code>lazyLoading</code> <code>imageFormat</code> <code>missingAlt</code>
Forms & identity	<code>missingForm</code> <code>formFields</code> <code>formLabels</code> <code>formIdentity</code> <code>forbiddenBusinessName</code> <code>missingBusinessName</code> <code>missingPhone</code> <code>phoneMismatch</code> <code>missingEmail</code> <code>emailDomain</code> <code>privacyPolicy</code> <code>logoLink</code>
Layout	<code>horizontalOverflow</code> <code>missingElement</code> <code>elementOverlap</code> <code>clippedText</code> <code>containerWidth</code> <code>pageGutter</code> <code>visualRegression</code> <code>typography</code> <code>brandColor</code>
Interaction	<code>interaction</code> <code>hoverState</code> <code>focusIndicator</code>
SEO	<code>seoTitle</code> <code>seoDescription</code> <code>seoH1</code> <code>seoNoindex</code> <code>seoViewport</code> <code>seoLang</code> <code>seoCanonical</code> <code>seoOpenGraph</code> <code>seoMultipleH1</code> <code>headingSkip</code> <code>robotsMissing</code> <code>robotsBlocked</code> <code>sitemap</code>
Accessibility	<code>accessibilityCritical</code> <code>accessibilityMinor</code>
Runtime	<code>pageError</code> <code>consoleError</code> <code>failedRequest</code> <code>crossBrowser</code> <code>lighthouse</code> <code>moduleCrash</code>

4.13 Environment variables

VARIABLE	EFFECT
<code>QA_BASE_URL</code>	Overrides <code>baseUr1</code> . CI sets <code>http://127.0.0.1:4321</code> . Use it to run against staging.
<code>QA_CONFIG</code>	Path to a different config file — useful for multi-site monorepos.
<code>QA_REUSE_CRAWL=0</code>	Force a fresh crawl instead of reusing <code>qa-results/crawl.json</code> .
<code>QA_FAIL_ON_WARNINGS=1</code>	Make warnings block deployment too.
<code>QA_CHROMIUM_CHANNEL</code>	Run against an installed Chrome or Edge locally. Leave unset in CI.

5 Getting to a clean first run

The first run on a real site will produce findings. That is the point. The work is deciding, for each one, which of three things it is.

THE FINDING IS...	ACTION
A real bug	Fix the site. This is the system doing its job.
A real rule you are not ready to enforce yet	Move it to <code>"warning"</code> or <code>"off"</code> in <code>severity</code> , with a note to revisit.
A misconfiguration	Fix <code>qa-config.json</code> — usually a selector that does not match.

What not to do is disable rules wholesale to get to green. The following seven are the ones that actually prevent embarrassment, and should stay at **ERROR** from day one:

RULE	WHAT IT STOPS REACHING PRODUCTION
<code>forbiddenBusinessName</code>	Another client's name on this client's site
<code>internalLink</code>	Navigation to a page that does not exist
<code>missingForm</code>	A contact page that cannot capture a lead
<code>brokenRequiredPage</code>	A core page returning 404 or 500
<code>horizontalOverflow</code>	A site that scrolls sideways on a phone
<code>seoNoindex</code>	A staging <code>noindex</code> shipped to production
<code>privacyPolicy</code>	A site with no reachable privacy policy

A reasonable day-one starting point

```
"severity": {
  "accessibilityMinor": "off",
  "lazyLoading": "warning",
  "typography": "warning",
  "externalLink": "off"
}
```

Target zero errors. Carrying warnings is fine and expected — they are the backlog.

EXPECT A SECOND ROUND IN CI

CI is a different machine. Different fonts, a slower Lighthouse, a cold cache. A few new findings on the first CI run is normal, not a sign that something is wrong. Fix or tune, push again.

Commit and push

```
git add .
git commit -m "Add automated website QA CI/CD"
git push -u origin setup/qa-cicd
```

6 Phase 2A — GitLab implementation

The engine has nothing platform-specific in it, so GitLab is a good place to trial the system before committing to it everywhere. Only `.gitlab-ci.yml` differs from the GitHub setup.

5 Add the CI/CD variables

Settings → CI/CD → Variables → Add variable, three times:

KEY	VALUE	FLAGS
<code>CLOUDFLARE_API_TOKEN</code>	Scoped token (section 8)	Masked, Protected
<code>CLOUDFLARE_ACCOUNT_ID</code>	From Workers & Pages → Overview	Masked, Protected
<code>CLOUDFLARE_PROJECT</code>	Pages project name, exactly	Protected

DO THIS BEFORE YOU PUSH

Protected restricts a variable to jobs running on protected branches. Without it, a merge request from a feature branch or a fork can read your Cloudflare token. Protect the branch first (next step), then the variables work on `main` and nowhere else.

Leave `CLOUDFLARE_PROJECT` unmasked — masking imposes length and character rules that project names often fail, and the name is not sensitive.

6 Protect the default branch

Settings → Repository → Protected branches

- Branch: `main`
- Allowed to push: **No one**
- Allowed to merge: Maintainers, or Developers + Maintainers
- Allowed to force push: off

Settings → Merge requests

- Enable **Pipelines must succeed**
- Optionally, **All threads must be resolved**

"Pipelines must succeed" is GitLab's equivalent of a required status check. Because the reporter exits 1 on any error, a failing QA run fails the job, fails the pipeline and blocks the merge.

7 Push and inspect

Push the branch and open a merge request. Check three places:

- **CI/CD → Pipelines** — the `qa` job runs. The `deploy` job does not appear for a merge request.
- **The merge request page** — a test summary listing every check, from the JUnit report. Errors appear as failures, warnings as skipped.
- **Job → Browse artifacts** — `qa-report.md`, screenshots, Lighthouse HTML.

Runner notes

The pipeline needs a Docker-executor runner, which is the default on GitLab.com shared runners. It uses `node:20-bookworm` and installs Playwright browsers with `--with-deps`, caching them in `.playwright/` keyed on `package-lock.json`. Expect 8–15 minutes for a first run and 4–8 minutes afterwards.

A faster image. Microsoft's Playwright image ships the browsers preinstalled:

```
variables:
  NODE_IMAGE: "mcr.microsoft.com/playwright:v1.49.0-noble"
```

Then delete the `npx playwright install` line. The image tag must match the Playwright version in your `package-lock.json`, or you get a confusing "Executable doesn't exist" error. If you would rather not track that, keep the default.

Shared runner minutes. On GitLab.com's free tier, running Playwright and Lighthouse on every push to every branch consumes compute minutes quickly. To trial it more cheaply, narrow the trigger:

```
qa:
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
```

Self-hosted runners have no such limit; run it on everything.

Manual approval before production

GitLab's protected environments feature requires Premium. On Free, make the deploy job manual so a person clicks to release:

```
deploy:
  rules:
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH && $CI_PIPELINE_SOURCE == "push"
      when: manual
```

7 Phase 2B — GitHub implementation

Everything tuned on GitLab carries over unchanged. In each repo you swap the CI file, move three variables and replace the branch rules.

5 Open the pull request and watch the first run

GitHub → Pull requests → New pull request → base `main`, compare `setup/qa-cicd`.

In the Actions tab, the QA and Deploy run executes the `Website QA` job. The `Deploy to Cloudflare Pages` job shows as `skipped` — correct, because pull requests never deploy.

IF THE RUN IS BLOCKED

Organisations often restrict which actions may run. Fix at **Settings** → **Actions** → **General** → **Actions permissions**, allowing at minimum `actions/checkout`, `actions/setup-node`, `actions/cache` and `actions/upload-artifact`.

6 Read the results

- **Job summary** — bottom of the run page. The whole `qa-report.md` renders there, tables and all. This is the link to send a project manager.
- **Annotations** — errors and warnings appear inline at the top of the run and against the pull request.
- **Artifacts** — `qa-report-<run id>`, containing the reports, screenshots, Lighthouse HTML and `preview.log`. Uploaded even when QA fails.

7 Add the secrets and the variable

Settings → Secrets and variables → Actions

TAB	NAME	VALUE
Secrets	<code>CLOUDFLARE_API_TOKEN</code>	Scoped token (section 8)
Secrets	<code>CLOUDFLARE_ACCOUNT_ID</code>	From Workers & Pages → Overview
Variables	<code>CLOUDFLARE_PROJECT</code>	Pages project name, exactly

The project name is a variable rather than a secret deliberately: it is not sensitive, and keeping it readable makes a misconfiguration obvious. The deploy job verifies all three before attempting anything and fails with an annotation naming whichever is missing.

8 Create the ruleset

Settings → Rules → Rulesets → New ruleset → New branch ruleset

- Name: `Protect main`
- Enforcement status: **Active**
- Target branches: Add target → **Include default branch**

Enable: **Require a pull request before merging**; **Require status checks to pass** → Add checks → **Website QA**; **Block force pushes**.

ORDERING MATTERS

`Website QA` only appears in the status-check picker *after* it has reported at least once. Run the pull request first, then create the ruleset. The name comes from the job's `name:` in the workflow — if you rename the job, update the ruleset or the rule silently stops protecting anything.

9 Optional — require a human for production

Settings → Environments → **production** → **Required reviewers**. The deploy job already runs in the `production` environment, so adding reviewers means every production deploy waits for a named person to approve, even after QA passes. Worth it for a high-stakes site; unnecessary friction for a brochure site.

8 Phase 3 — Cloudflare

Create the Pages project

If the site is already on Cloudflare Pages, note the exact project name — it is case-sensitive. Otherwise create a **Direct Upload** project. Do not connect it to Git; the pipeline does the deploying.

```
npx wrangler pages project create acme-plumbing --production-branch=main
```

Create a least-privilege API token

Cloudflare dashboard → **My Profile** → **API Tokens** → **Create Token**. Use the **Edit Cloudflare Workers** template, or build a custom token with:

- **Permissions:** Account → *Cloudflare Pages* → **Edit**
- **Account Resources:** Include → your account only

Copy it immediately; Cloudflare shows it once.

NEVER USE THE GLOBAL API KEY

It grants full access to every zone on the account and cannot be scoped down. A scoped Pages token can do exactly one thing: deploy to Pages.

Find the account ID

Workers & Pages → **Overview**; the Account ID is in the right-hand sidebar. It is also the hex string in the dashboard URL after `dash.cloudflare.com/`.

Prevent double deployment

THE STEP PEOPLE SKIP

If Cloudflare Pages is connected directly to the repository — it supports both GitHub and GitLab — Cloudflare builds and deploys on its own whenever the default branch changes, **without ever looking at your QA result**. You would have a gate that blocks nothing while appearing to work.

Option A — CI owns production (recommended). Cloudflare dashboard → **Workers & Pages** → your project → **Settings** → **Builds & deployments** → **Disconnect** the Git integration. The project, its custom domain and its deployment history all remain; only the automatic Git build stops. `wrangler pages deploy` keeps working through the Direct Upload path. From that point nothing reaches production without passing QA.

Option B — Cloudflare owns production, QA stays advisory. Keep the Git integration, delete the deploy job, and rely on branch protection: QA must pass before a merge, and only merges trigger Cloudflare's build. Weaker, because an admin pushing straight to the default branch bypasses QA entirely. Choose this only if something else depends on Cloudflare's own build pipeline.

Output directory

The deploy step runs `wrangler pages deploy dist`. If `astro.config.mjs` sets a different `outDir`, change `dist` in the CI file to match.

9 Phase 4 — Proving the gate works

Do not trust a gate you have not watched fail. Run these on the setup branch before merging; each costs about three minutes of CI. Revert each one once you have seen it work.

#	BREAK THIS	EXPECTED RESULT
1	Push an ordinary change	QA runs; deploy skipped
2	Add <code>x</code>	FAIL naming the broken target and the page linking to it
3	Point an <code></code> at a missing file	FAIL or WARN per config, naming the image and page
4	Visit any invalid route in a browser	HTTP 404 with header, navigation, content and footer intact
5	Add <code><div style="width:1800px">x</div></code>	FAIL at mobile and tablet, plus a screenshot artifact
6	Delete the form from <code>/contact/</code>	FAIL — "Required form not found"
7	Put a <code>forbiddenBusinessNames</code> value on a page	FAIL naming the value and the page
8	Break the mobile menu toggle	FAIL from the interaction check
9	Remove a <code><label></code> from a form field	axe reports it at the configured severity
10	Set <code>lighthouse.performance</code> to <code>0.99</code>	FAIL ; deploy does not run

ALREADY VERIFIED

Rows 2–10 were each exercised against a fixture site while the engine was built. Broken links, broken images, the 404 probe, 1,441 px of mobile overflow, a removed form, a leaked company name, a dead menu toggle, an axe `image-alt` violation and an unreachable Lighthouse threshold all produced the messages above, and the reporter exited 1 every time. A clean run on the same fixture gave 51 passes, 0 errors and exit 0.

What a passing run looks like

```
=====
WEBSITE QA REPORT
=====
[PASS] Required page
[PASS] SEO basics
[PASS] Browser runtime
[PASS] Contact form
[PASS] Form fields
[PASS] Typography
[PASS] Logo link
[PASS] Privacy policy
[PASS] Phone number
[PASS] Email address
[PASS] 404 page
[PASS] robots.txt
[PASS] Sitemap
[PASS] Internal links
[PASS] Broken images
```

```
[PASS] Image alt text
[PASS] Layout
[PASS] Interaction
[PASS] Hover state
[PASS] Focus indicator
[PASS] Accessibility
[PASS] Cross-browser
[PASS] Lighthouse

0 error(s), 0 warning(s), 51 passed check(s)

QA PASSED
```

And a failing one

```
[FAIL] Business name (1 error)
[FAIL] Contact form (1 error)
[FAIL] Internal link (1 error)
[FAIL] Horizontal overflow (4 errors)
[WARN] Broken image (1 warning)
[WARN] Image alt text (1 warning)

6 error(s), 5 warning(s), 45 passed check(s)

QA FAILED - deployment blocked
```

With, in the log and against the pull request:

```
::error title=Pages / SEO / Identity: Business name::Forbidden business name
"Old Client Company" found on the page [/]
::error title=Links: Internal link::/this-page-does-not-exist/ returned HTTP 404
- linked from / [/]
::error title=Responsive Layout: Horizontal overflow::mobile: page scrolls 1441px
horizontally - div (right 1816px) [/ mobile]
```

10 Day-to-day workflow

- Developer branches and pushes
- QA runs automatically — every branch, every push
 - Fail — developer reads the report, fixes, pushes again
 - Pass — the merge/pull request can be reviewed and approved
- Merge to the default branch
- QA runs again on the merged result
 - Pass → Cloudflare production deployment
 - Fail → deployment never starts

THE RULE

QA runs on every push. Deployment does not. Production deploys only from the default branch, and only after QA passes.

Who reads what

ROLE	LOOKS AT
Developer	The annotations or merge-request test list; screenshots for layout failures; the full job log only when something is genuinely odd
Reviewer	The status check or pipeline result — it is binary, and it blocks the merge
Project manager	<code>qa-report.md</code> in the job summary or artifacts: plain-language tables of what passed, what warned and what failed

11 Rolling it out across every site

A CI definition has to exist inside each repository — neither GitHub nor GitLab will run a pipeline that is not in the repo. But "a file" can mean fifteen lines rather than twenty files.

MODEL	FILES PER REPO	UPDATING THE ENGINE MEANS
Copy the kit What the installer does today	About 20	Re-running the installer with <code>--force</code> on every repo
Central kit repo Reusable workflow or <code>include:</code>	2 — a short CI file and <code>qa-config.json</code>	One push to the kit repo; every site picks it up on its next run
Private npm package	3 — CI file, config, one dependency	A version bump per repo

For a team running many client sites, the central kit repo is the right answer. Each site repo then holds only:

```
.github/workflows/qa.yml    (or .gitlab-ci.yml)    ~15 lines, calls the shared definition
scripts/qa-config.json      the client's name, phone, pages, selectors
```

WHY THIS WORKS WITHOUT CODE CHANGES

The engine resolves every path from the current working directory, and Node resolves `import 'playwright'` relative to the importing file. So the shared engine can be checked out into `.qa-engine/` and run against the site at the repo root — and the QA dependencies stay in the kit repo, leaving each client's `package.json` untouched. Nothing in `scripts/` needs to change to move between models.

Two things to arrange first:

- **GitHub:** if the kit repo is private, set **Settings** → **Actions** → **General** → **Access** on it to allow access from other repositories in the organisation, and give the calling workflow a token that can check it out.
- **GitLab:** use `include: project:` pointing at the kit project; the same visibility consideration applies.

Until then, each additional site is

```
git checkout -b setup/qa-cicd
node /path/to/astro-qa-kit/install-qa.mjs .
npm install -D playwright @playwright/test @lhci/cli wait-on wrangler @axe-core/playwright
```

Then edit `qa-config.json`, add the three variables, set the branch rules, and merge. The engine files are byte-identical across every site; only the config differs.

12 Troubleshooting

SYMPTOM	CAUSE AND FIX
<code>browserType.launch:</code> Executable doesn't exist	Playwright browsers are missing or the cache is stale. Confirm the cache key includes <code>package-lock.json</code> so a Playwright upgrade re-downloads them. On GitLab, clear runner caches and re-run. If you switched to the Playwright Docker image, its tag must match your installed Playwright version.
<code>wait-on</code> times out	The preview server never came up. Read <code>preview.log</code> in the artifacts. SSR sites cannot use <code>astro preview</code> — serve with the adapter's command and set <code>QA_BASE_URL</code> .
Every external link fails, but only in CI	Third-party hosts rate-limit CI runners. 401, 403, 405, 429, 503 and 999 are already skipped. Set <code>severity.externalLink: "off"</code> if it is still noisy.
Lighthouse scores move between runs	Shared runner CPU. Raise <code>lighthouse.numberOfRuns</code> to 3 (the median is used) or lower the performance threshold. Keep accessibility, best practices and SEO strict — those are stable.
"No Lighthouse results found"	<code>lhci autorun</code> did not finish its upload step, so <code>.lighthouseci/manifest.json</code> was never written. The step runs with <code>continue-on-error</code> , so read its log.
Dozens of accessibility findings	Set <code>severity.accessibilityMinor: "off"</code> , fix critical and serious first, or list specific rule ids in <code>accessibility.disableRules</code> .
"Trigger not found" on interactions	Your selectors differ from the config. Add <code>data-qa</code> attributes, or set <code>"optional": true</code> while the component is being built.
The crawl is slow or the job times out	Lower <code>crawl.maxPages</code> , add <code>crawl.excludePatterns</code> , shorten <code>responsivePages</code> and <code>lighthousePages</code> , reduce <code>imageRules.maxPages</code> . The crawl runs once and is shared by every module.
Deploy job cannot see the Cloudflare token (GitLab)	The variable is Protected but the branch is not. Protect the branch, then re-run.
Two pipelines fire per push (GitLab)	The <code>workflow:rules</code> block at the top of <code>.gitlab-ci.yml</code> prevents this — make sure it was copied.
Merge request shows no test report (GitLab)	The <code>qa</code> job must upload <code>qa-report.junit.xml</code> . <code>artifacts: when: always</code> should upload it even on failure — check the job's artifact list.
Website QA missing from the status-check picker (GitHub)	The check appears only after it has reported once. Run the pull request first, then create the ruleset.
<code>wrangler</code> authentication error	The token lacks <i>Cloudflare Pages: Edit</i> , or the account ID belongs to a different account. Confirm the project name matches exactly, including case.
The site deployed even though QA failed	Cloudflare's Git integration is still connected. See section 8, "Prevent double deployment".

Adding a new check later

Create `scripts/<name>-check.mjs`, wrap the body in `runModule('<name>', async (cfg, results) => { ... })`, record findings with `results.issue(cfg, '<ruleKey>', 'Check name', 'message', { page })`, add a default

severity to `DEFAULT_SEVERITY` in `qa-lib.mjs`, and add a step to the CI file. The reporter picks it up automatically — it reads whatever JSON files exist in `qa-results/`.

A Appendix A — every check

#	CHECK	DEFAULT
1	Required pages load; 4xx, 5xx and timeouts fail, redirects allowed	ERROR
2	Custom 404 page returns 404 and keeps header, nav, content, footer, assets and layout	ERROR
3	Internal links resolve, including soft-404 detection	ERROR
4	External links reachable; rate-limited hosts skipped	WARNING
5	External links use <code>target="_blank"</code> (opt-in)	WARNING
6	Broken images: failed loads, zero natural size, broken background images	ERROR critical WARNING other
7	Image distortion from a stretched aspect ratio	WARNING
8	A form exists on every configured form page	ERROR
9	Form required fields and submit button present	ERROR
10	Form fields have accessible labels	WARNING
11	No wrong company name inside a form region	ERROR
12	No forbidden business name in text, title, meta, Open Graph or footer	ERROR
13	Expected business name present on required pages	WARNING
14	Phone present in text and <code>tel:</code> links	ERROR
15	Visible phone and <code>tel:</code> link agree	WARNING
16	Email present in text and <code>mailto:</code> links	ERROR
17	<code>mailto:</code> links use the expected domain	WARNING
18	Privacy policy page exists and is linked from header, nav or footer	ERROR
19	Logo is visible, clickable and links to <code>/</code>	ERROR
20	No horizontal overflow at any of five viewports	ERROR
21	Required elements stay visible at every viewport	ERROR
22	No clipped text or element overlap	WARNING
23	Container max-width and page gutters	WARNING
24	Visual regression against approved baselines (opt-in)	WARNING
25	Typography: family, size, weight, line-height per selector	WARNING
26	Brand colours on key selectors	WARNING
27	Heading hierarchy: missing H1	ERROR
28	Heading hierarchy: multiple H1 or skipped levels	WARNING

#	CHECK	DEFAULT
29	Hover, focus and keyboard-focus states produce a visible change	WARNING
30	Menus, modals, dropdowns, accordions, tabs and drawers open and close	ERROR
31	Image optimisation: file size, lazy loading, modern formats	WARNING
32	SEO: title, meta description, H1, viewport meta, <code>lang</code> , accidental <code>noindex</code>	ERROR
33	SEO: canonical and Open Graph metadata	WARNING
34	Missing <code>alt</code> attributes and unlabelled icon buttons	WARNING
35	axe-core WCAG 2.1 A/AA scan	ERROR critical/serious WARNING other
36	<code>robots.txt</code> exists and does not block the whole site	ERROR if blocked
37	Sitemap exists	WARNING
38	Uncaught JavaScript errors	ERROR
39	Console errors and failed same-origin requests	WARNING
40	Cross-browser smoke suite in Firefox and WebKit	ERROR
41	Lighthouse: Performance ≥ 75 , Accessibility ≥ 90 , Best Practices ≥ 90 , SEO ≥ 90	ERROR

ON WEBKIT AND SAFARI

Playwright's WebKit is an automated approximation of Safari's engine, not Safari itself. It catches the large majority of Safari-specific layout and interaction problems. For a release-critical launch, a final check on a real iOS device is still worth the ten minutes.

B Appendix B — the complete qa-config.json

The shipped defaults. Copy this into a new site and change the identity, pages and selectors.

```
{
  "baseUrl": "http://localhost:4321",

  "businessName": "Nextkin",
  "businessNameAliases": [],
  "forbiddenBusinessNames": ["Old Client Company"],

  "phone": "+1 604 555 1234",
  "requiredPhone": true,
  "email": "hello@nextkin.com",
  "emailDomain": "nextkin.com",
  "requiredEmail": true,

  "privacyPolicyUrl": "/privacy-policy/",
  "requiredPages": ["/", "/about/", "/services/", "/contact/", "/privacy-policy/"],

  "formPages": ["/contact/"],
  "formSelectors": ["form", "iframe[src*='leadconnectorhq']", "iframe[src*='hubspot']"],
  "formRequiredFields": ["name", "email"],

  "logoSelector": "header a",
  "headerSelector": "header",
  "footerSelector": "footer",
  "navSelector": "header nav",
  "notFoundPath": "/qa-404-probe-do-not-create/",

  "crawl": {
    "enabled": true,
    "maxPages": 100,
    "concurrency": 5,
    "checkExternal": true,
    "requireExternalBlank": false,
    "excludePatterns": ["/admin", "/api/"],
    "softNotFoundText": []
  },

  "viewports": {
    "mobile": { "width": 375, "height": 812 },
    "tablet": { "width": 768, "height": 1024 },
    "laptop": { "width": 1366, "height": 768 },
    "desktop": { "width": 1920, "height": 1080 },
    "wide": { "width": 2560, "height": 1440 }
  },
  "responsivePages": ["/", "/contact/"],

  "design": {
    "containerSelector": null,
    "containerMaxWidth": 1280,
    "pageGutterMin": 16,
    "overflowTolerancePx": 2,
    "requiredVisibleSelectors": ["header", "main", "footer"],
    "noOverlapSelectors": [],
    "brandColors": ["#111827", "#2563EB"],
    "colorRules": [],
    "typography": {
      "body": { "selector": "body", "fontFamily": "Inter", "fontSize": "16px", "lineHeight": "24px" }
```

```

    }
  },

  "states": {
    "enabled": true,
    "pages": ["/"],
    "selectors": ["header nav a", "a.btn", "button[type='submit']"],
    "maxElementsPerSelector": 5,
    "requireFocusIndicator": true
  },

  "interactions": [
    {
      "name": "Mobile navigation",
      "page": "/",
      "viewport": "mobile",
      "trigger": "[data-qa='menu-toggle'], button[aria-label*='menu' i], .hamburger, .menu-toggle",
      "target": "[data-qa='mobile-menu'], #mobile-menu, .mobile-nav",
      "expect": "visible",
      "closeAfter": true,
      "optional": true
    }
  ],

  "accessibility": {
    "enabled": true,
    "pages": ["/", "/contact/"],
    "tags": ["wcag2a", "wcag2aa", "wcag21a", "wcag21aa"],
    "disableRules": [],
    "errorImpacts": ["critical", "serious"]
  },

  "imageRules": {
    "maxFileSizeKB": 500,
    "requireLazyLoadBelowFold": true,
    "preferModernFormats": false,
    "checkBackgroundImages": true,
    "aspectRatioTolerance": 0.1,
    "maxPages": 50,
    "criticalSelectors": ["header img", "img[alt*='logo' i]", "[class*='hero'] img"]
  },

  "visualRegression": {
    "enabled": false,
    "maxDiffRatio": 0.01,
    "pages": ["/"],
    "viewports": ["mobile", "desktop"]
  },

  "browsers": ["chromium", "firefox", "webkit"],
  "crossBrowser": {
    "pages": ["/", "/contact/"],
    "viewports": ["mobile", "desktop"]
  },

  "runtime": {
    "ignorePatterns": [
      "favicon.ico",
      "google-analytics",
      "googletagmanager",
      "leadconnectorhq",
      "hubspot",
      "ResizeObserver loop"
    ]
  }
}

```

```

    ]
  },

  "seo": {
    "checkAllCrawledPages": true,
    "pages": [],
    "requireCanonical": true,
    "requireOpenGraph": true
  },

  "robots": { "enabled": true },
  "sitemap": { "enabled": true, "paths": ["/sitemap-index.xml", "/sitemap.xml"] },

  "lighthousePages": ["/", "/about/", "/services/", "/contact/"],
  "lighthouse": {
    "enabled": true,
    "numberOfRuns": 2,
    "performance": 0.75,
    "accessibility": 0.9,
    "bestPractices": 0.9,
    "seo": 0.9
  },

  "timeouts": { "navigationMs": 30000, "requestMs": 15000 },

  "severity": {}
}

```